Organização e Arquitetura de Computadores

Processadores

Quelita Ribeiro

Departamento de Computação / UFS

Roteiro

- ✓Paralelismo no nível de instrução
- ✓ Paralelismo no nível de processador
- ✓Computadores paralelos
- ✓ Multiprocessadores
- ✓ Multicomputadores

Paralelismo no nível de instrução

- Grande parte dos arquitetos de computadores busca o paralelismo
  - fazer duas ou mais coisas ao mesmo tempo
- como um meio de conseguir desempenho melhor para dada velocidade de clock.

- O paralelismo tem duas formas gerais:

- **no nível de instrução e**
- **no nível de processador.**

Paralelismo no nível de instrução

- O paralelismo tem duas formas gerais: **no nível de instrução e no nível de processador.**

- **No nível de instrução**, o paralelismo é explorado dentro de instruções individuais para obter da máquina mais instruções por segundo.
- **No nível de processador**, várias CPUs trabalham juntas no mesmo problema.

- Cada abordagem tem seus próprios méritos.

Paralelismo no nível de instrução

Paralelismo no nível de instrução

- **Pipelining (paralelismo)**

- o processo de buscar instruções na memória é um grande **gargalo** na velocidade de execução da instrução.

Paralelismo no nível de instrução

- **Pipelining (paralelismo)**

- Para amenizar essa situação havia a possibilidade de buscar instruções na memória antecipadamente, de maneira que estivessem presentes quando necessárias.

- Essas instruções eram armazenadas em um conjunto de registradores denominado **buffer de busca antecipada (ou prefetch buffer).**

Paralelismo no nível de instrução

- A busca antecipada divide a execução da instrução em duas partes:

- **a busca e**
- **a execução propriamente dita.**

- O conceito de pipeline (paralelismo) amplia muito essa estratégia.

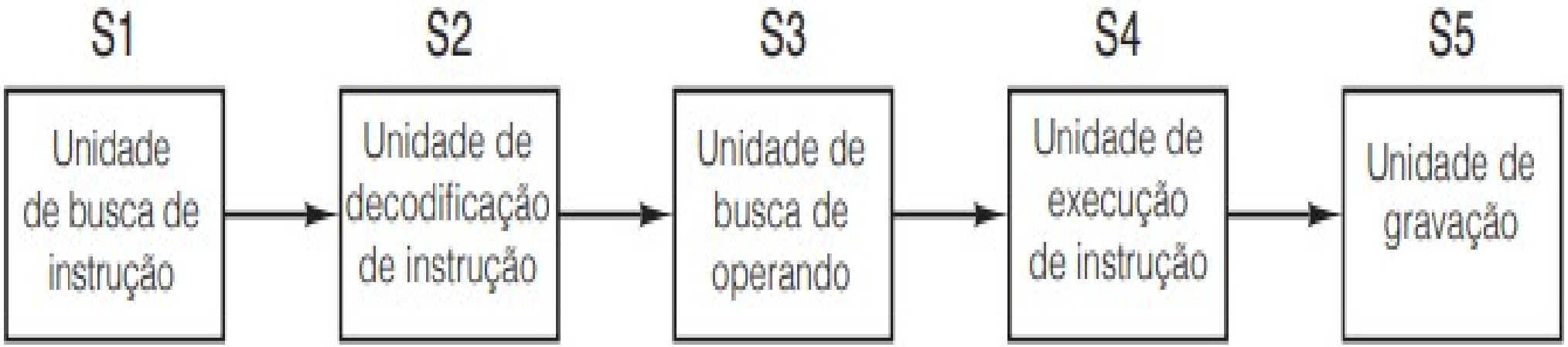
Paralelismo no nível de instrução

- Em vez de dividir a execução da instrução em apenas duas partes, muitas vezes ela é dividida em muitas partes (uma dúzia ou mais),
- cada uma manipulada por uma parte dedicada do hardware, e todas elas podem executar em paralelo.

Paralelismo no nível de instrução

- A Figura ilustra um pipeline com cinco unidades, também denominadas estágios.

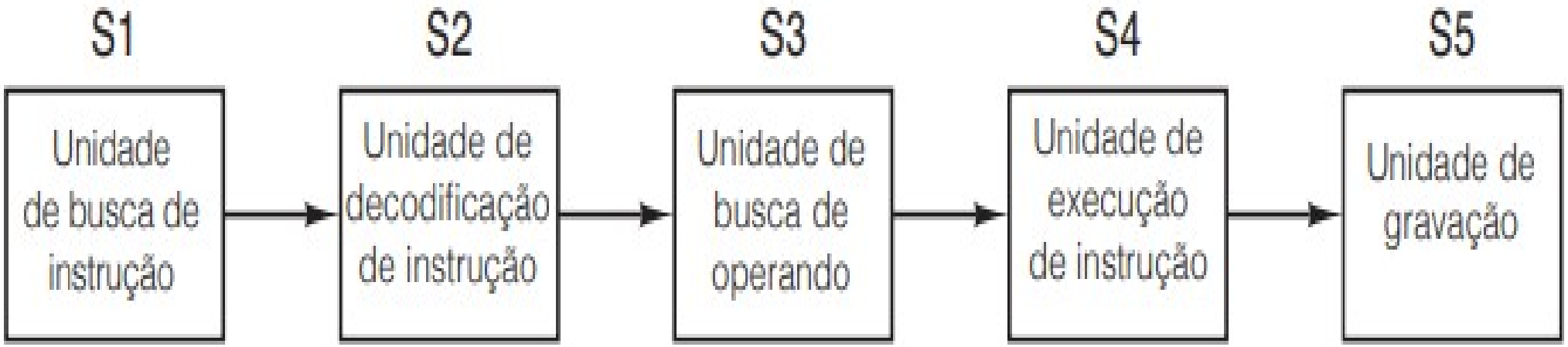
- O estágio 1 busca a instrução na memória e a coloca em um buffer até que ela seja necessária.



(a)

Paralelismo no nível de instrução

- O estágio 2 decodifica a instrução
- O estágio 3 localiza e busca os operandos, seja nos registradores, seja na memória.

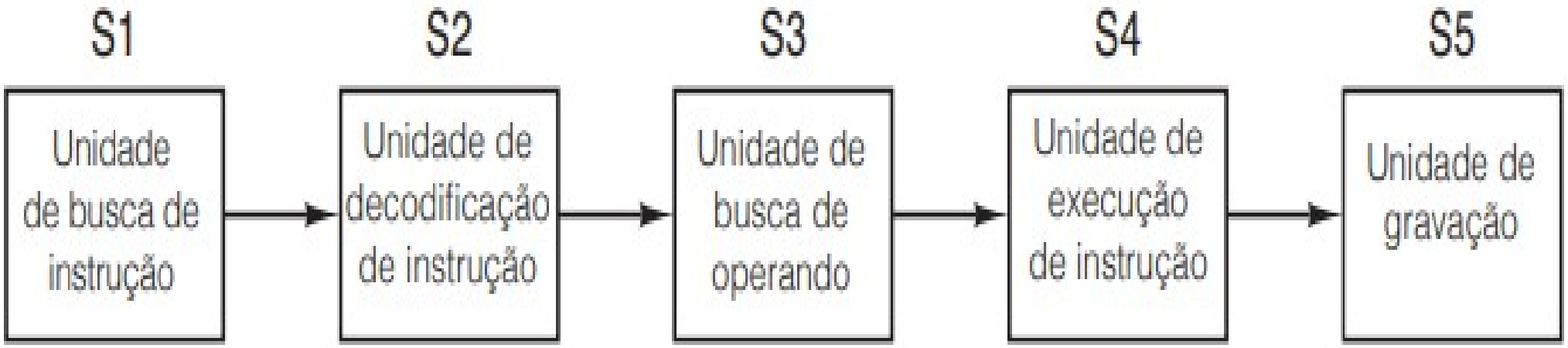


(a)

Paralelismo no nível de instrução

- O estágio 4 é que realiza o trabalho de executar a instrução

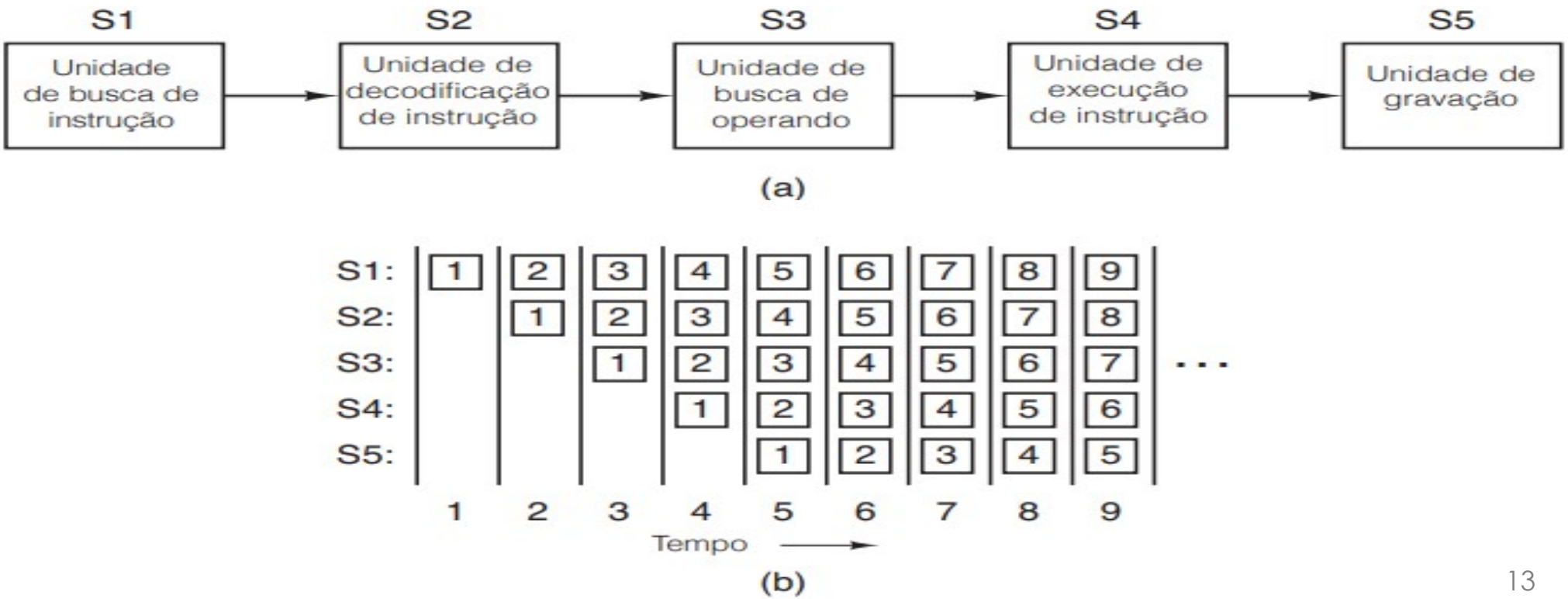
- Por fim, o estágio 5 escreve o resultado de volta no registrador adequado.



(a)

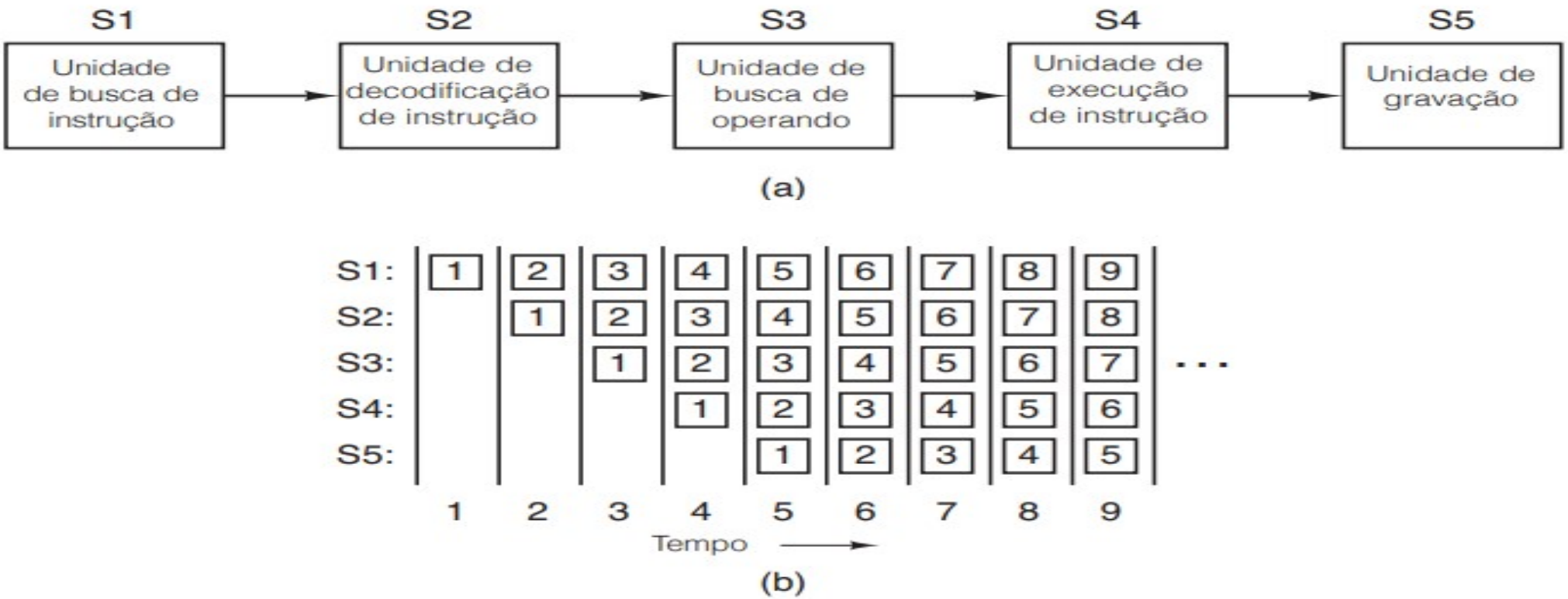
Paralelismo no nível de instrução

- Na Figura, vemos o pipeline funcionando em função do tempo.
- Durante o ciclo de clock 1, o estágio S1 está **buscando a instrução 1 na memória**.
- Durante o ciclo 2, o **S2 decodifica a instrução 1**, enquanto o **S1 busca a instrução 2**.



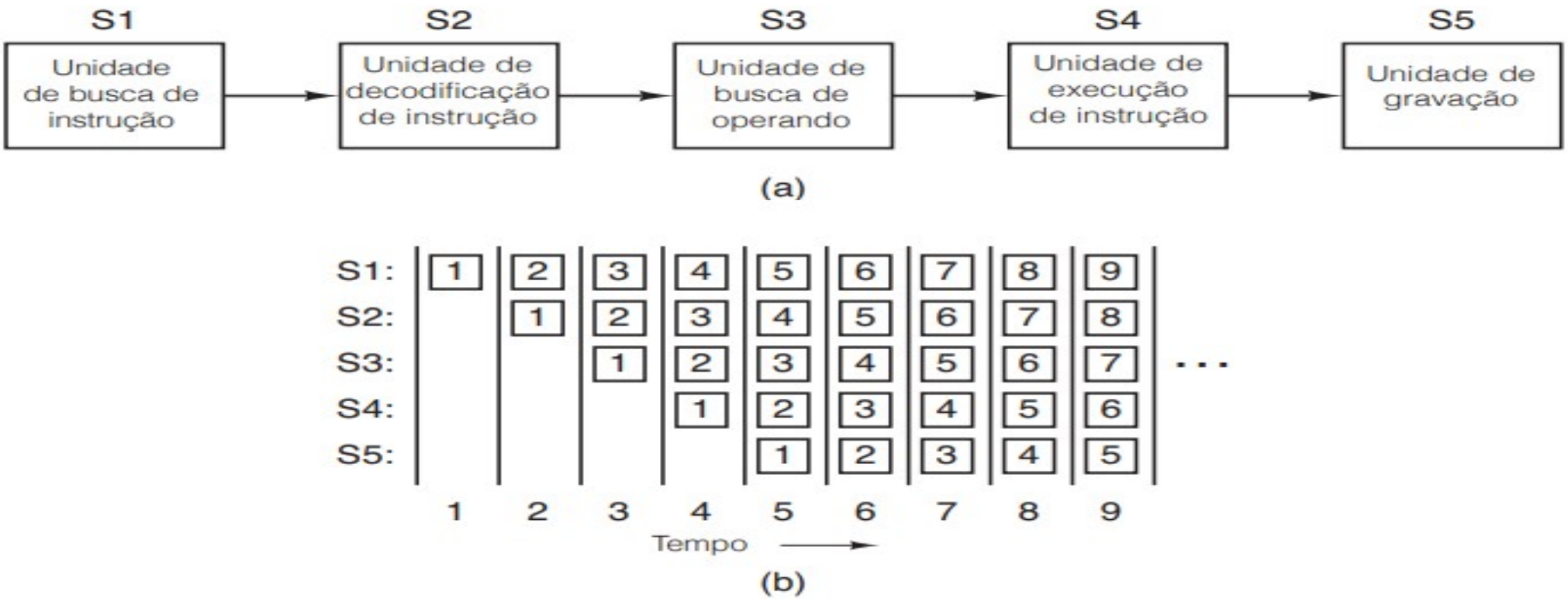
Paralelismo no nível de instrução

- Durante o ciclo 3, o **S3 busca os operandos para a instrução 1**, o **S2 decodifica a instrução 2** e o **S1 busca a terceira instrução**.



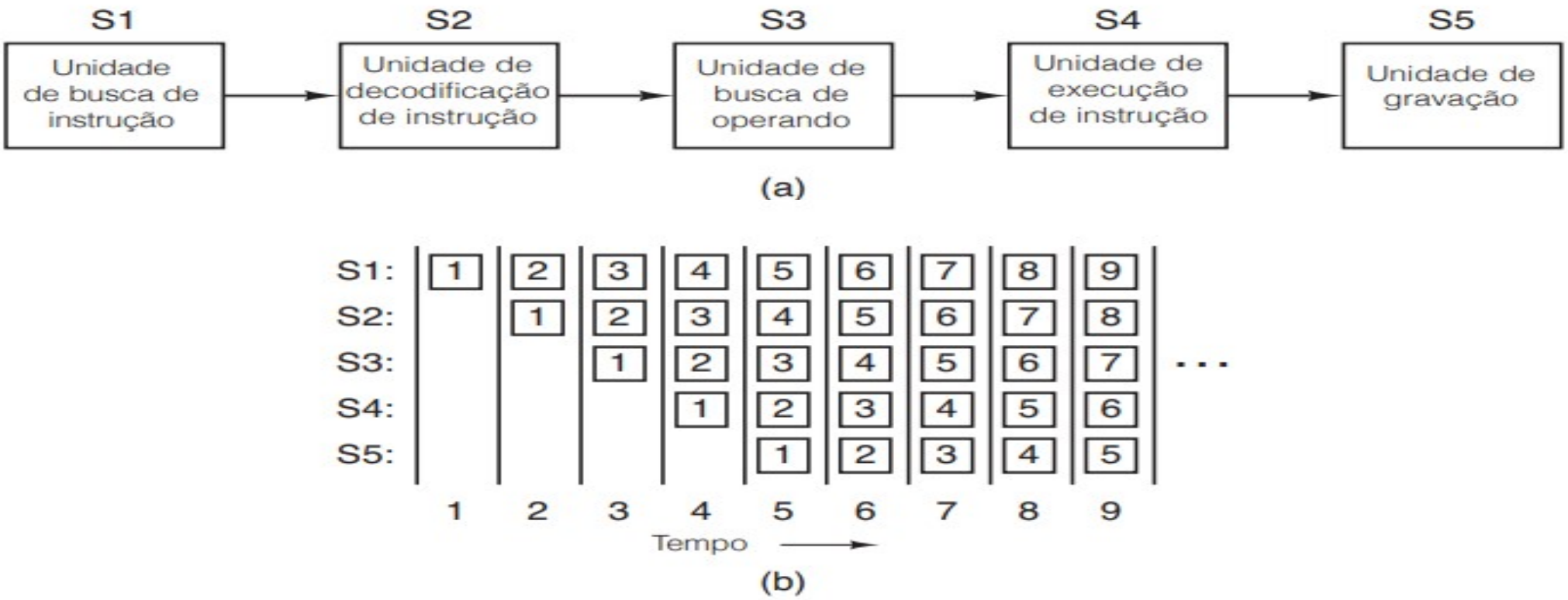
Paralelismo no nível de instrução

- Durante o ciclo 4, o **S4 executa a instrução 1**, **S3 busca os operandos para a instrução 2**, **S2 decodifica a instrução 3** e **S1 busca a instrução 4**.



Paralelismo no nível de instrução

- Por fim, durante o ciclo 5, **S5 escreve (grava) o resultado da instrução 1** de volta ao registrador, enquanto os outros estágios trabalham nas instruções seguintes.



Paralelismo no nível de instrução

- Suponha que o tempo de ciclo dessa máquina seja 2 ns.
- Sendo assim, uma instrução leva 10 ns para percorrer todo o caminho do pipeline de cinco estágios.

Paralelismo no nível de instrução

- À primeira vista, como uma instrução demora 10 ns, parece que a máquina poderia funcionar em 100 MIPS, mas, na verdade, ela funciona muito melhor do que isso.

- A cada ciclo de clock (2 ns), uma nova instrução é concluída, portanto, a velocidade real de processamento é 500 MIPS, e não 100 MIPS.

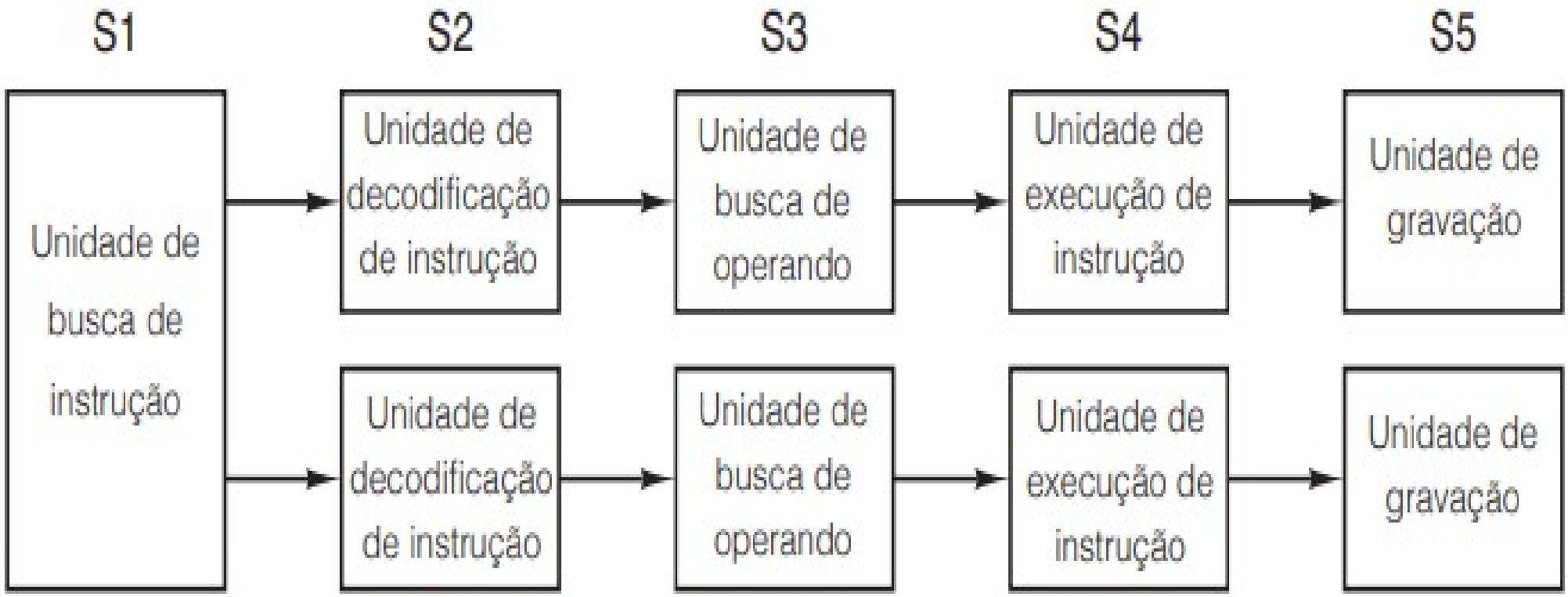
Arquiteturas superescalares

Arquiteturas superescalares

- Se um pipeline é bom, então certamente dois pipelines são ainda melhores.

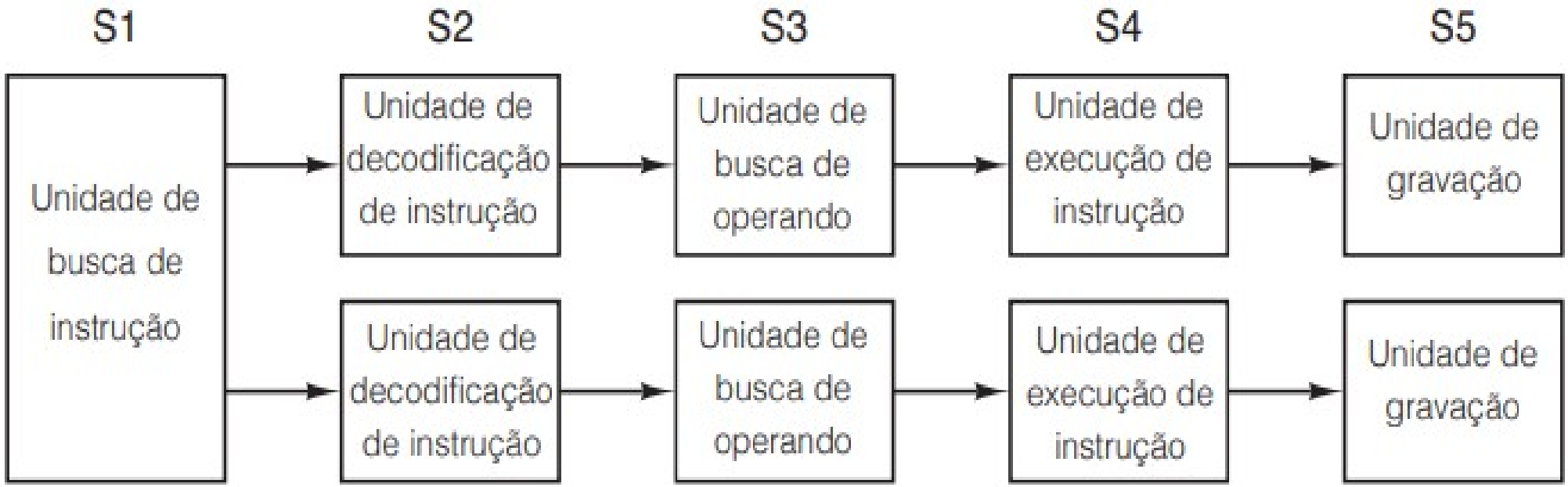
Arquiteturas superescalares

- Um projeto possível para uma CPU com dois pipelines é mostrado na Figura.



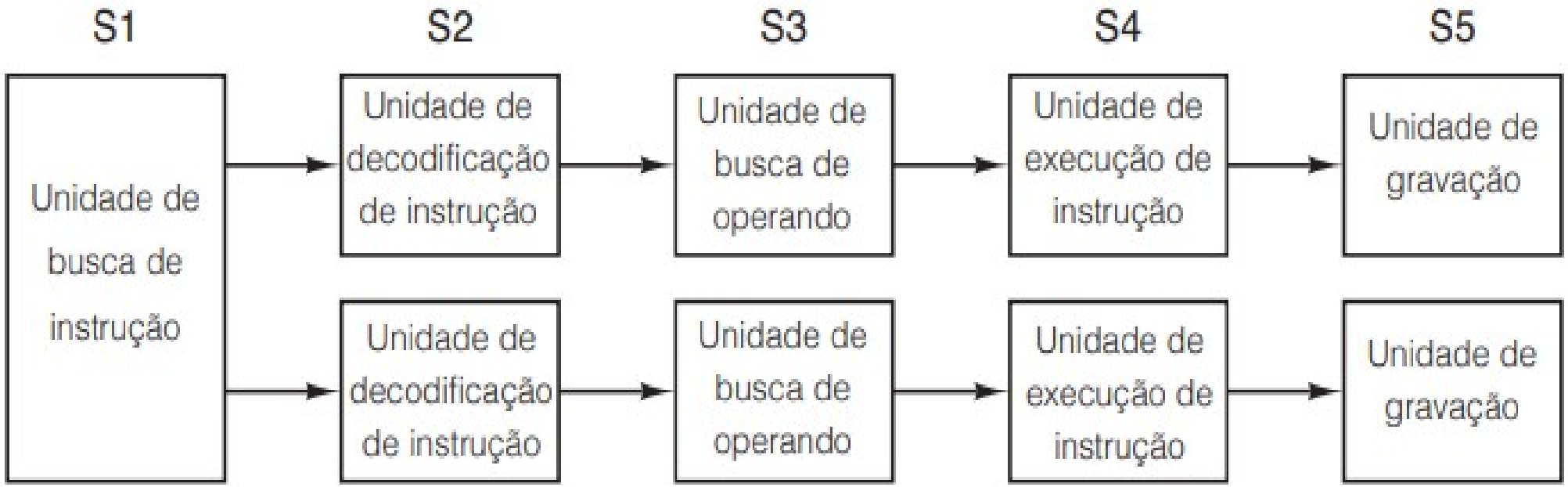
Arquiteturas superescalares

- Nesse caso, uma **única** unidade de **busca de instruções** busca pares de instruções ao mesmo tempo e coloca cada uma delas em seu próprio pipeline, completo com sua própria ULA para operação paralela.



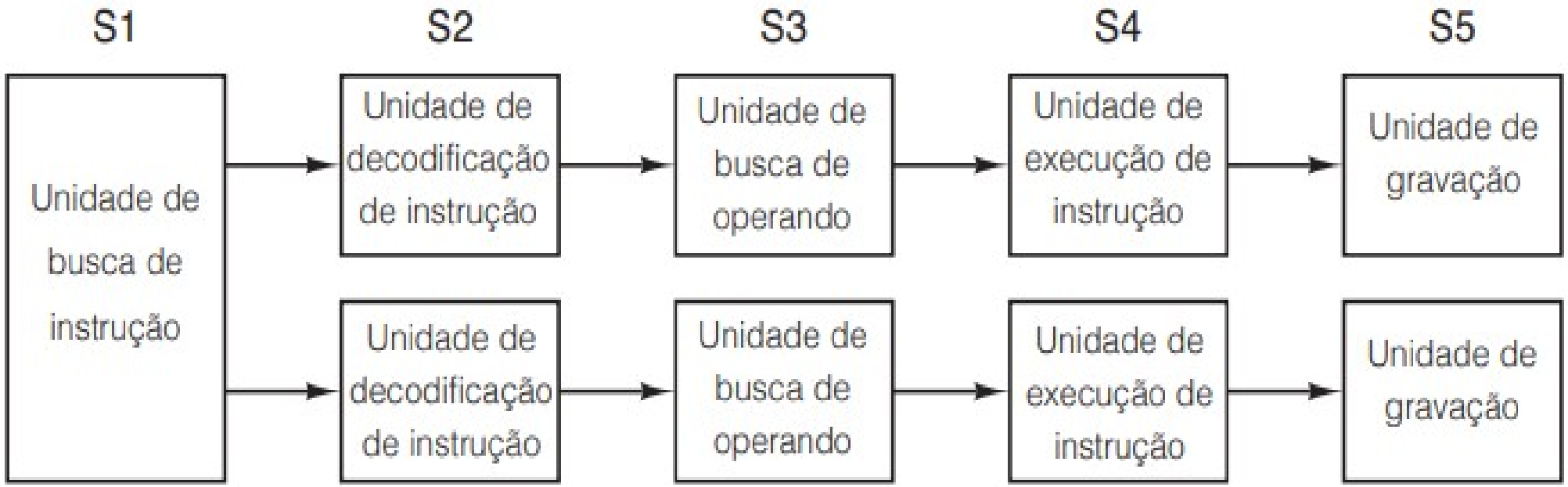
Arquiteturas superescalares

- Para poder executar em paralelo, as duas instruções não devem ter conflito de utilização de recursos (por exemplo, registradores) e nenhuma deve depender do resultado da outra.



Arquiteturas superescalares

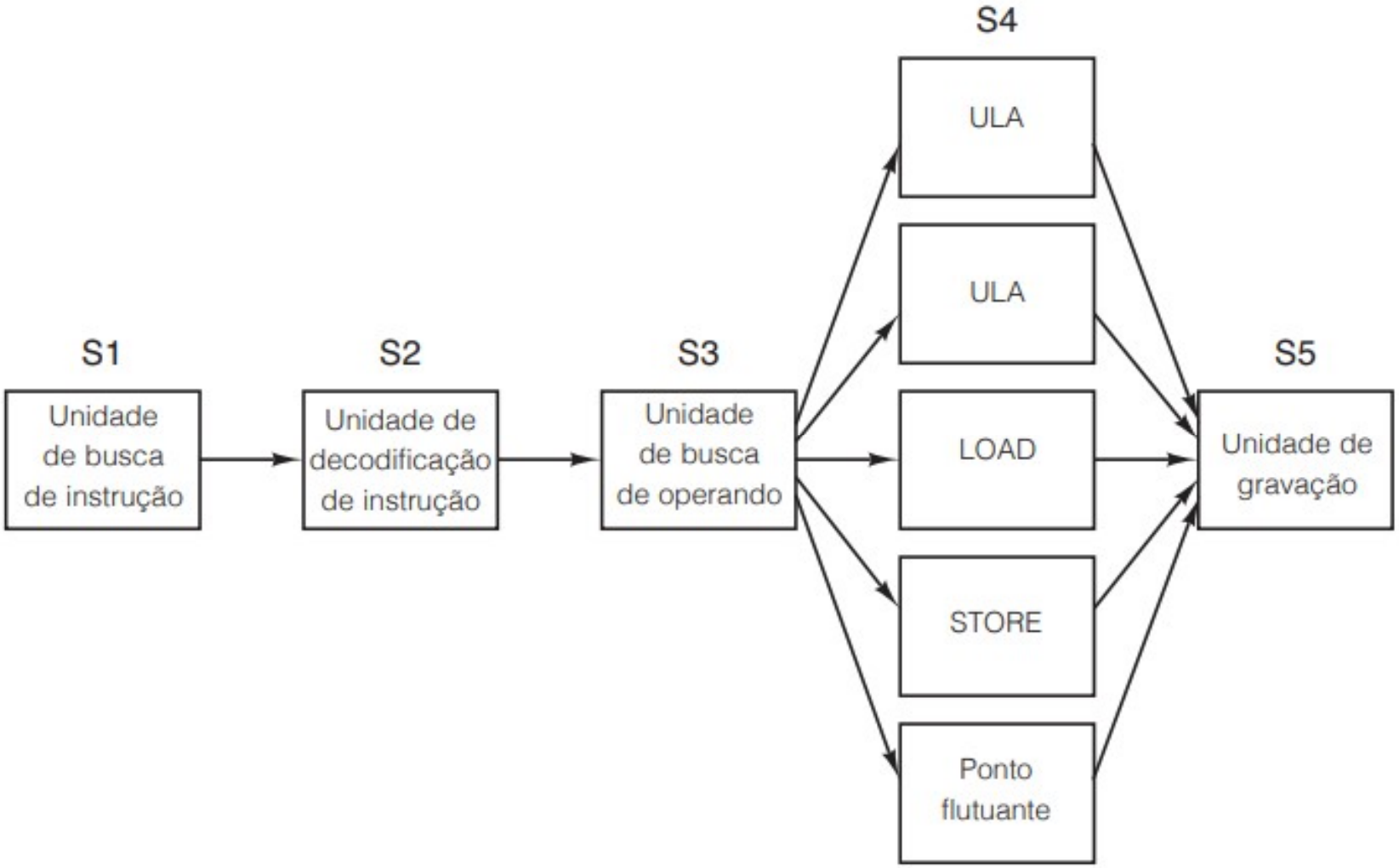
- Os conflitos deverão ser detectados e eliminados durante a execução usando hardware extra.



Arquiteturas superescalares

- Evolução:

Figura 2.6 Processador superescalar com cinco unidades funcionais.



Paralelismo no nível do processador

Paralelismo no nível do processador

- Paralelismo no nível de instrução ajuda,
- Mas pipelining e operação superescalar rendem um fator de cinco ou dez de ganho em desempenho.
- **Para obter ganhos de 50, 100 ou mais, a única maneira é projetar computadores com várias CPUs;**

Computadores paralelos

Computadores paralelos

- Necessário para um número substancial de problemas em domínios de cálculo como
  - *físicas,*
  - *engenharia e*
  - *gráficos de computador* envolvendo **laços e matrizes**

- Pois, muitas vezes, **os mesmos cálculos são efetuados em muitos conjuntos diferentes de dados ao mesmo tempo.**

Computadores paralelos

- A regularidade e a estrutura desses programas os tornam alvos especialmente fáceis para aceleração por meio de execução paralela.

Computadores paralelos

- Há dois métodos que têm sido usados para executar esses programas altamente regulares de modo rápido e eficaz:
- **processadores SIMD** e
- **Processadores vetoriais.**

Computadores paralelos

- Embora esses dois esquemas guardem semelhanças em seus aspectos,
- **O processador SIMD** é considerado um computador paralelo,
- enquanto o **Processador vetorial** é considerado uma extensão de um processador único.

Computadores paralelos

- **Como todos os processadores estão rodando a mesma instrução, o sistema só precisa de um "cérebro" controlando o computador.**

- Em consequência, o processador só precisa de:

- um estágio de busca,
- um estágio de decodificação e
- um conjunto de lógica de controle.

Computadores paralelos

- Essa é uma enorme economia no silício, que dá aos computadores paralelos uma grande vantagem sobre outros processadores,
- Desde que o software que eles estejam rodando seja **altamente regular, com bastante paralelismo.**

Computadores paralelos

- **Um processador SIMD**
  - *Single Instruction-stream Multiple Data-stream* ou
  - fluxo único de instruções, fluxo múltiplo de dados

- **consiste em um grande número de processadores idênticos que efetuam a mesma sequência de instruções sobre diferentes conjuntos de dados.**

Multiprocessadores

Multiprocessadores

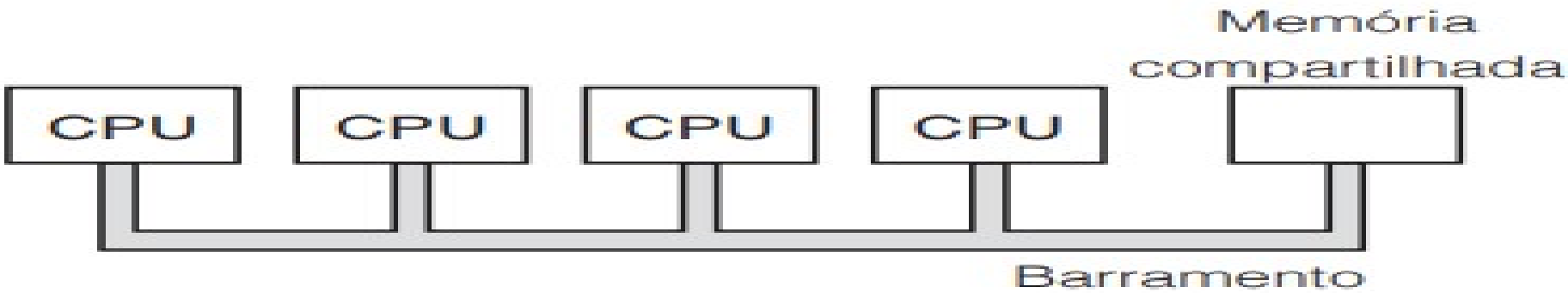
- Os elementos de processamento em um **processador SIMD** não são CPUs independentes, **uma vez que há uma só unidade de controle compartilhada por todos eles.**

Multiprocessadores

- **O primeiro sistema paralelo com CPUs totalmente desenvolvidas é o multiprocessador**, um sistema com mais de uma CPU que compartilha uma memória em comum.

Multiprocessadores

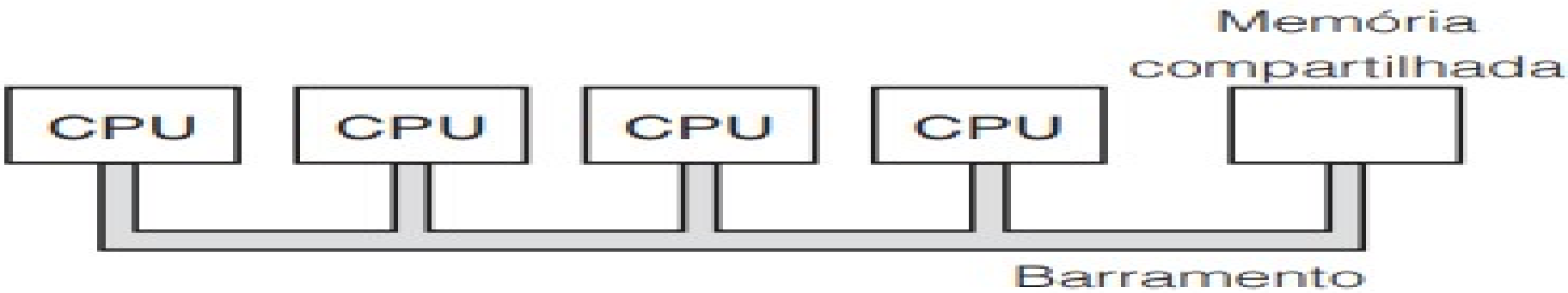
- Uma vez que cada CPU pode ler ou escrever em qualquer parte da memória, elas devem se coordenar (em software) para evitar que uma atrapalhe a outra.



(a)

Multiprocessadores

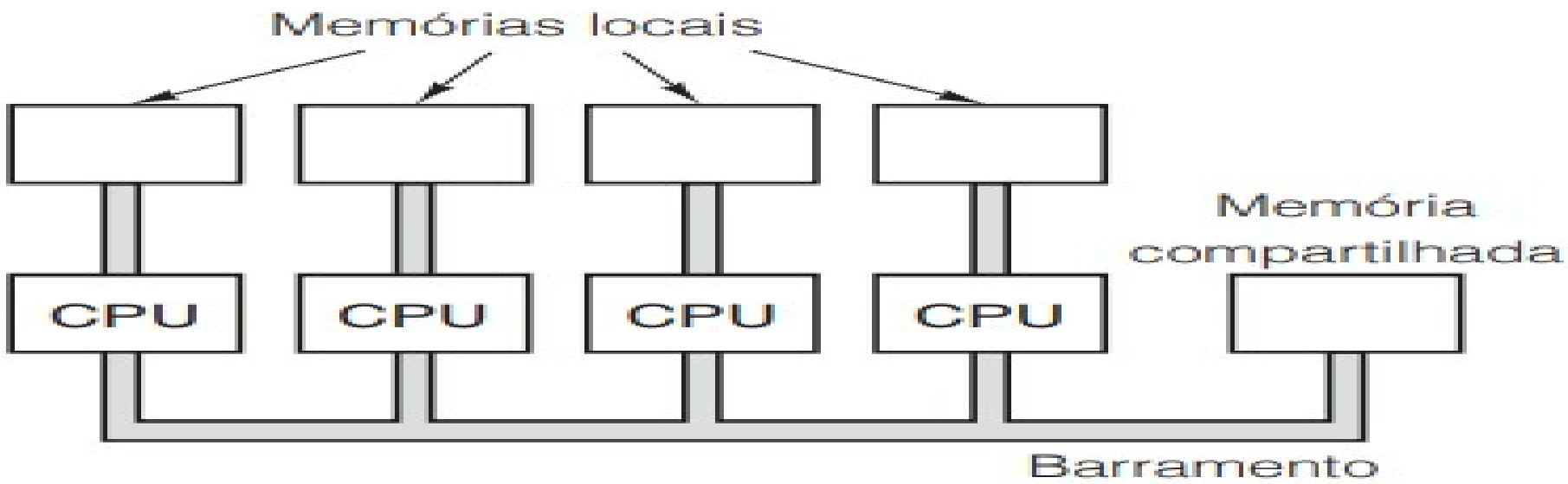
- Há vários esquemas de implementação possíveis.
- O mais simples é um barramento único com várias CPUs e uma memória, todas ligadas nele: Figura a.



(a)

Multiprocessadores

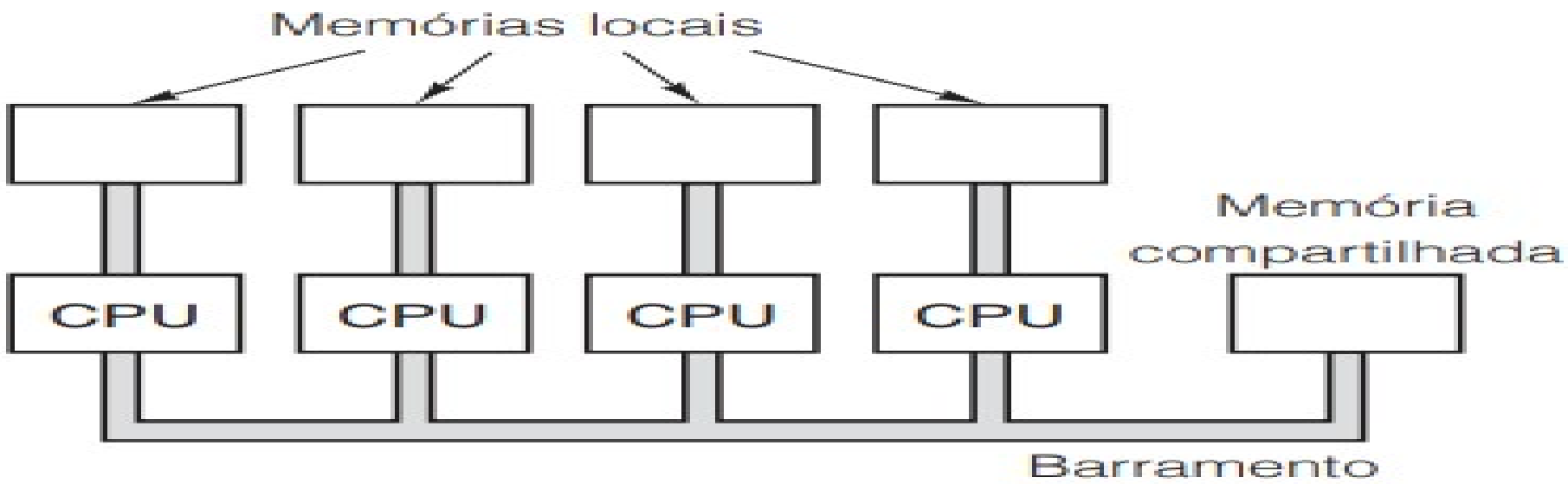
- Porém, surgem conflitos com um grande número de processadores velozes tentando acessar a memória pelo **mesmo barramento**.
- Outro projeto é dá a cada processador um pouco de memória local só dele, que não é acessível para os outros: Figura(b).



(b)

Multiprocessadores

- Essa memória pode ser usada para o código de programa e para os itens de dados que não precisam ser compartilhados.
- O acesso a essa memória privada não usa o barramento principal, o que reduz muito o tráfego no barramento.



(b)

Multicomputadores

Multicomputadores

- Embora seja um tanto fácil construir multiprocessadores com um número modesto de processadores (≤ 256),
- construir grandes é surpreendentemente difícil.
- A dificuldade está em conectar todos os processadores à memória.

Multicomputadores

- Para evitar esses problemas, muitos projetistas simplesmente abandonaram a ideia de ter uma memória compartilhada e passaram a construir sistemas que consistissem em grandes números de computadores interconectados, cada um com sua própria memória privada, mas nenhuma em comum.
- Esses sistemas são denominados **multicomputadores**.

Multicomputadores

- As CPUs de um multicomputador se comunicam enviando mensagens umas às outras, mais ou menos como enviar e-mails, porém, com muito mais rapidez.

- O resultado é que mensagens de um computador para outro muitas vezes passam por um ou mais computadores ou comutadores (chaves) intermediários para ir da fonte até o destino.

Referência

TANENBAUM, Andrew S.; ZUCCHI, Wagner Luiz. **Organização estruturada de computadores**. Pearson Prentice Hall, 2013.